1)Demonstrate the use of Group by and Order by clause.

```
CREATE TABLE Department
(
  Deptno    INT PRIMARY KEY,
  DeptName  VARCHAR2(30),
  Location  VARCHAR2(30),
  Post VARCHAR2(30)
);

INSERT INTO Department VALUES (1, 'MCA', 'Jalgaon', 'HOD');
INSERT INTO Department VALUES (2, 'BCA', 'Jalgaon', 'HOD');
INSERT INTO Department VALUES (3, 'BTech', 'Jalgaon', 'HOD');
INSERT INTO Department VALUES (4, 'MTech', 'Jalgaon', 'HOD');
INSERT INTO Department VALUES (5, 'BBA', 'Jalgaon', 'HOD');
INSERT INTO Department VALUES (6, 'MBA', 'Jalgaon', 'HOD');
INSERT INTO Department VALUES (7, 'MCA', 'Amravati', 'HOD');
INSERT INTO Department VALUES (8, 'BCA', ' Amravati ', 'HOD');
INSERT INTO Department VALUES (9, 'MCA', 'Nagpur', 'HOD');
INSERT INTO Department VALUES (10, 'BCA', 'Nagpur', 'HOD');
INSERT INTO Department VALUES (11, 'MCA', 'Pune', 'HOD');
INSERT INTO Department VALUES (12, 'BCA', ' Pune', 'HOD');
INSERT INTO Department VALUES (13, 'BTech', 'Pune', 'HOD');
INSERT INTO Department VALUES (14, 'MTech', 'Pune', 'HOD');

select *from Department
```

Results | Explain | Describe | Saved SQL | History

DEPTNO	DEPTNAME	LOCATION	POST
1	MCA	Jalgaon	HOD
2	BCA	Jalgaon	HOD
3	BTech	Jalgaon	HOD
4	MTech	Jalgaon	HOD
5	BBA	Jalgaon	HOD
6	MBA	Jalgaon	HOD
7	MCA	Amravati	HOD
8	BCA	Amravati	HOD
9	MCA	Nagpur	HOD
10	BCA	Nagpur	HOD
11	MCA	Pune	HOD
12	BCA	Pune	HOD
13	BTech	Pune	HOD
14	MTech	Pune	HOD

14 rows returned in 0.00 seconds CSV Export

Query:-

```
SELECT Location, COUNT(*) AS Total_Departments
FROM Department
GROUP BY Location
ORDER BY Total_Departments DESC;
```

Results | Explain | Describe | Saved SQL | History

LOCATION	TOTAL_DEPARTMENTS
Jalgaon	6
Pune	3
Nagpur	2
Amravati	1
Amravati	1
Pune	1

2) Consider the following schema for a Hospital Database:
DOCTOR (Did, Dname, DAddress, Qualification)
PATIENTMASTER (Pcode, Pname, Padd, age, gender, bloodgroup, Pid)
ADMMITTEDPATIENT(Pcode, EntryDate, DischargeDate, WardNo, Disease)
a) Find the detail of the doctor who is treating the patient of ward no3
b) Find the detail of patient who are admitted within period 03/03/2020 to 25/05/2020
c) Find the detail of patient who are suffered from blood cancer
d) Create view on DOCTOR & PATIENTASTER tables.

1.DOCTOR TABLE:-

```
CREATE TABLE Doctor(
Did NUMBER PRIMARY KEY,
Dname VARCHAR2(100),
DAddress VARCHAR2(100),
Qualification VARCHAR2(100));

INSERT INTO Doctor VALUES(101,'Dr.Ramesh','Jalgaon','MBBS');
INSERT INTO Doctor VALUES(102,'Dr.A.H.Mali','Jalgaon','MD');
INSERT INTO Doctor VALUES(103,'Dr.D.N.Patil','Pachora','MD');
INSERT INTO Doctor VALUES(104,'Dr.Nikita','Bhusawal','MBBS');
INSERT INTO Doctor VALUES(105,'Dr.P.D.Mali','Dhule','MBBS');

select *from Doctor
```

Results | Explain | Describe | Saved SQL | History

DID	DNAME	DADDRESS	QUALIFICATION
101	Dr.Ramesh	Jalgaon	MBBS
102	Dr.A.H.Mali	Jalgaon	MD
103	Dr.D.N.Patil	Pachora	MD
104	Dr.Nikita	Bhusawal	MBBS
105	Dr.P.D.Mali	Dhule	MBBS

2.PATIENTMASTER TABLE:-

```
CREATE TABLE PATIENTMASTER(
Pcode NUMBER PRIMARY KEY,
Pname VARCHAR2(50),
Paddresss VARCHAR2(50),
age NUMBER(3),
gender VARCHAR2(7),
bloodgroup VARCHAR2(5),
Pid NUMBER);

INSERT INTO PATIENTMASTER VALUES(201,'Nikita','Bhusawal',21,'F','A+',101);
INSERT INTO PATIENTMASTER VALUES(202,'Nida','Dharangaon',20,'F','A-',102);
INSERT INTO PATIENTMASTER VALUES(203,'Pranali','Varangaon',20,'F','AB+',103);
```

INSERT INTO PATIENTMASTER VALUES(204,'Neha','Jalgaon',19,'F','O+',104);
INSERT INTO PATIENTMASTER VALUES(205,'Mamta','Jalgaon',18,'F','A+',105);

select *from PATIENTMASTER

Results Explain Describe Saved SQL History

PCODE	PNAME	PADDRESSS	AGE	GENDER	BLOODGROUP	PID
201	Nikita	Bhusawal	21	F	A+	101
202	Nida	Dharangaon	20	F	A-	102
203	Pranali	Varangaon	20	F	AB+	103
204	Neha	Jalgaon	19	F	O+	104
205	Mamta	Jalgaon	18	F	A+	105

5 rows returned in 0.00 seconds CSV Export

3.ADMMITTEDPATIENT:-

```
CREATE TABLE ADMMITTEDPATIENT(
Did NUMBER,
Pcode NUMBER,
EntryDate DATE,
DischargeDate DATE,
WardNo NUMBER,
Disease VARCHAR2(50),
CONSTRAINT fk_admmitted_patient
  FOREIGN KEY (Pcode)REFERENCES PATIENTMASTER(Pcode),
CONSTRAINT fk_admmitted_doctor
  FOREIGN KEY (Did)REFERENCES Doctor(Did)
);
INSERT INTO ADMMITTEDPATIENT VALUES(101,201,TO_DATE('2026-02-12','YYYY-MM-DD'),TO_DATE('2026-02-16','YYYY-MM-DD'),1,'Sinus');
INSERT INTO ADMMITTEDPATIENT VALUES(102,202,TO_DATE('2026-02-22','YYYY-MM-DD'),TO_DATE('2026-02-28','YYYY-MM-DD'),4,'Blood Cancer');
INSERT INTO ADMMITTEDPATIENT VALUES(103,203,TO_DATE('2026-03-05','YYYY-MM-DD'),TO_DATE('2026-03-09','YYYY-MM-DD'),7,'Blood Cancer');
INSERT INTO ADMMITTEDPATIENT VALUES(104,204,TO_DATE('2026-04-12','YYYY-MM-DD'),TO_DATE('2026-04-15','YYYY-MM-DD'),3,'Asthma');
INSERT INTO ADMMITTEDPATIENT VALUES(105,205,TO_DATE('2026-05-15','YYYY-MM-DD'),TO_DATE('2026-05-20','YYYY-MM-DD'),5,'Migraine');

select *from ADMMITTEDPATIENT
```

Results Explain Describe Saved SQL History

DID	PCODE	ENTRYDATE	DISCHARGEDATE	WARDNO	DISEASE
101	201	12-FEB-26	16-FEB-26	1	Sinus
102	202	22-FEB-26	28-FEB-26	4	Blood Cancer
103	203	05-MAR-26	09-MAR-26	7	Blood Cancer
104	204	12-APR-26	15-APR-26	3	Asthma
105	205	15-MAY-26	20-MAY-26	5	Migraine

QUERY:-

a) Find the detail of the doctor who is treating the patient of ward no3

```
SELECT D.*
FROM DOCTOR D
JOIN ADMMITTEDPATIENT A ON D.Did = A.Did
WHERE A.WardNo = 3;
```

Results Explain Describe Saved SQL History

DID	DNAME	DADDRESS	QUALIFICATION
104	Dr.Nikita	Bhusawal	MBBS

b) Find the detail of patient who are admitted within period 03/03/2020 to 25/05/2020

```
SELECT P.*
FROM PATIENTMASTER P
JOIN ADMMITTEDPATIENT A ON P.Pcode = A.Pcode
WHERE A.EntryDate BETWEEN
TO_DATE('2026-03-03','YYYY-MM-DD') AND
TO_DATE('2026-05-25','YYYY-MM-DD');
```

Results Explain Describe Saved SQL History

PCODE	PNAME	PADDRESSS	AGE	GENDER	BLOODGROUP	PID
203	Pranali	Varangaon	20	F	AB+	103
204	Neha	Jalgaon	19	F	O+	104
205	Mamta	Jalgaon	18	F	A+	105

3 rows returned in 0.02 seconds CSV Export

c) Find the detail of patient who are suffered from blood cancer

```
SELECT P.*
FROM PATIENTMASTER P
JOIN ADMMITTEDPATIENT A ON P.Pcode = A.Pcode
WHERE A.Disease = 'Blood Cancer';
```

Results Explain Describe Saved SQL History

PCODE	PNAME	PADDRESSS	AGE	GENDER	BLOODGROUP	PID
202	Nida	Dharangaon	20	F	A-	102
203	Pranali	Varangaon	20	F	AB+	103

2 rows returned in 0.01 seconds CSV Export

d) Create view on DOCTOR & PATIENTASTER tables.

```
CREATE OR REPLACE VIEW DoctorPatientView AS
SELECT D.Did,D.Dname,D.Qualification,P.Pname,Pcode FROM Doctor D
LEFT JOIN PATIENTMASTER P
ON D.Did=P.Pid;
```

Results Explain Describe Saved SQL History

```
View created.
```

select *from DoctorPatientView

Results Explain Describe Saved SQL History

DID	DNAME	QUALIFICATION	PNAME	PCODE
101	Dr.Ramesh	MBBS	Nikita	201
102	Dr.A.H.Mali	MD	Nida	202
103	Dr.D.N.Patil	MD	Pranali	203
104	Dr.Nikita	MBBS	Neha	204
105	Dr.P.D.Mali	MBBS	Mamta	205

5 rows returned in 0.00 seconds CSV Export

3) Create department table with the following structure

Field Name	Data Type
Deptno	Int
DeptName	Varchar(30)
Location	Varchar(30)

a) Add column designation to the department table.
b) Insert values into the table.
c) List the records of dept table grouped by deptno.
d) Update the record where deptno is 9.
e) Delete any column data from the table.

- **Table Creation:-**

```
CREATE TABLE Department(
  Deptno    INT PRIMARY KEY,
  DeptName  VARCHAR2(30),
  Location  VARCHAR2(30)
);
```

a) Add column designation to the department table.

```
ALTER TABLE Department
ADD designation VARCHAR2(30);
```

Results Explain Describe Saved SQL History

```
Table altered.
```

b) Insert values into the table.

```
INSERT INTO Department VALUES (1, 'MCA', 'Jalgaon', 'HOD');
INSERT INTO Department VALUES (2, 'BCA', 'Jalgaon', 'HOD');
INSERT INTO Department VALUES (3, 'MBA', 'Jalgaon', 'HOD');
INSERT INTO Department VALUES (4, 'MCA', 'Amravati', 'HOD');
INSERT INTO Department VALUES (5, 'MCA', 'Nagpur', 'HOD');
INSERT INTO Department VALUES (6, 'BCA', 'Nagpur', 'HOD');
INSERT INTO Department VALUES (7, 'MCA', 'Pune', 'HOD');
INSERT INTO Department VALUES (8, 'BCA', ' Pune', 'HOD');
INSERT INTO Department VALUES (9, 'BTech', 'Pune', 'HOD');
select *from Department
```

Results Explain Describe Saved SQL History

DEPTNO	DEPTNAME	LOCATION	DESIGNATION
1	MCA	Jalgaon	HOD
2	BCA	Jalgaon	HOD
3	MBA	Jalgaon	HOD
4	MCA	Amravati	HOD
5	MCA	Nagpur	HOD
6	BCA	Nagpur	HOD
7	MCA	Pune	HOD
8	BCA	Pune	HOD
9	BTech	Pune	HOD

c) List the records of dept table grouped by deptno.

```
SELECT Deptno, COUNT(*) AS Total_Departments
FROM Department
GROUP BY Deptno;
```

Results Explain Describe Saved SQL History

DEPTNO	TOTAL_DEPARTMENTS
1	1
2	1
3	1
4	1
5	1
6	1
7	1
8	1
9	1

9 rows returned in 0.00 seconds CSV Export

d) Update the record where deptno is 9.

```
UPDATE Department
SET DeptName = 'Human Resources',
   Location = 'Delhi',
   designation = 'Manager'
WHERE Deptno = 9;
SELECT *FROM Department
```

Results Explain Describe Saved SQL History

DEPTNO	DEPTNAME	LOCATION	DESIGNATION
1	MCA	Jalgaon	HOD
2	BCA	Jalgaon	HOD
3	MBA	Jalgaon	HOD
4	MCA	Amravati	HOD
5	MCA	Nagpur	HOD
6	BCA	Nagpur	HOD
7	MCA	Pune	HOD
8	BCA	Pune	HOD
9	Human Resources	Delhi	Manager

e) Delete any column data from the table.

```
UPDATE Department
SET designation = NULL;
SELECT *FROM Department
```

Results Explain Describe Saved SQL History

DEPTNO	DEPTNAME	LOCATION	DESIGNATION
1	MCA	Jalgaon	-
2	BCA	Jalgaon	-
3	MBA	Jalgaon	-
4	MCA	Amravati	-
5	MCA	Nagpur	-
6	BCA	Nagpur	-
7	MCA	Pune	-
8	BCA	Pune	-
9	Human Resources	Delhi	-

4)Create database using following schema. Apply Integrity Constraints and answer the following queries using SQL. DOCTOR (Did, Dname, Daddress, qualification)
PATIENT (Pid, Pname, age, gender).

Integrity Constraints:

1. The values of any attributes should not be null.
2. Did should be unique constraints.
3. Pid should be unique constraints.
4. Gender value should be M (male) or F (female).

Queries:

a) Insert at least 10 records in table.
b) Find details of all doctors.
c) Delete the records from DOCTOR where qualification is M.S
d) Find details of patient where age is less than 40.
e) Update the patient name where patient id is 5.

1)DOCTOR TABLE:

```
CREATE TABLE DOCTOR (
   Did INT PRIMARY KEY,
   Dname VARCHAR(50) NOT NULL,
   Daddress VARCHAR(100) NOT NULL,
   qualification VARCHAR(50) NOT NULL );
```

a) Insert at least 10 records in table.

```
INSERT INTO DOCTOR VALUES(1, 'Dr. Sharma', 'Delhi', 'MBBS');
INSERT INTO DOCTOR VALUES(2, 'Dr. Mehta', 'Mumbai', 'M.D');
INSERT INTO DOCTOR VALUES(3, 'Dr. Rao', 'Chennai', 'M.S');
INSERT INTO DOCTOR VALUES(4, 'Dr. Verma', 'Pune', 'MBBS');
INSERT INTO DOCTOR VALUES(5, 'Dr. Singh', 'Kolkata', 'M.S');
INSERT INTO DOCTOR VALUES(6, 'Dr. Patel', 'Ahmedabad', 'M.D');
INSERT INTO DOCTOR VALUES(7, 'Dr. Khan', 'Hyderabad', 'MBBS');
INSERT INTO DOCTOR VALUES(8, 'Dr. Das', 'Jaipur', 'M.S');
INSERT INTO DOCTOR VALUES(9, 'Dr. Joshi', 'Nagpur', 'M.D');
INSERT INTO DOCTOR VALUES(10, 'Dr. Iyer', 'Bangalore', 'MBBS');
```

2)PATIENT TABLE:

```
CREATE TABLE PATIENT (
   Pid INT PRIMARY KEY,
   Pname VARCHAR(50) NOT NULL,
   age INT NOT NULL,
   gender CHAR(1) NOT NULL CHECK (gender IN ('M','F')) );
```

a) Insert at least 10 records in table.

```
INSERT INTO PATIENT VALUES(1, 'Amit', 25, 'M');
INSERT INTO PATIENT VALUES(2, 'Priya', 30, 'F');
INSERT INTO PATIENT VALUES(3, 'Vijay', 45, 'M');
INSERT INTO PATIENT VALUES(4, 'Sneha', 38, 'F');
INSERT INTO PATIENT VALUES(5, 'Karan', 50, 'M');
INSERT INTO PATIENT VALUES(6, 'Neha', 28, 'F');
```

```
INSERT INTO PATIENT VALUES(7, 'Ajay', 60, 'M');
INSERT INTO PATIENT VALUES(8, 'Pooja', 35, 'F');
INSERT INTO PATIENT VALUES(9, 'Vikas', 42, 'M');
INSERT INTO PATIENT VALUES(10, 'Anjali', 22, 'F');
```

b) Find details of all doctors.

```
SELECT * FROM DOCTOR;
```

Results Explain Describe Saved SQL History

DID	DNAME	DADDRESS	QUALIFICATION
1	Dr. Sharma	Delhi	MBBS
2	Dr. Mehta	Mumbai	M.D
3	Dr. Rao	Chennai	M.S
4	Dr. Verma	Pune	MBBS
5	Dr. Singh	Kolkata	M.S
6	Dr. Patel	Ahmedabad	M.D
7	Dr. Khan	Hyderabad	MBBS
8	Dr. Das	Jaipur	M.S
9	Dr. Joshi	Nagpur	M.D
10	Dr. Iyer	Bangalore	MBBS

c) Delete the records from DOCTOR where qualification is M.S

```
DELETE FROM DOCTOR
WHERE qualification = 'M.S';
```

Results Explain Describe Saved SQL History

DID	DNAME	DADDRESS	QUALIFICATION
1	Dr. Sharma	Delhi	MBBS
2	Dr. Mehta	Mumbai	M.D
4	Dr. Verma	Pune	MBBS
6	Dr. Patel	Ahmedabad	M.D
7	Dr. Khan	Hyderabad	MBBS
9	Dr. Joshi	Nagpur	M.D
10	Dr. Iyer	Bangalore	MBBS

d) Find details of patient where age is less than 40.

```
SELECT * FROM PATIENT
WHERE age<40;
```

Results Explain Describe Saved SQL History

PID	PNAME	AGE	GENDER
1	Amit	25	M
2	Priya	30	F
4	Sneha	38	F
6	Neha	28	F
8	Pooja	35	F
10	Anjali	22	F

e) Update the patient name where patient id is 5.

```
UPDATE PATIENT
SET Pname = 'Rahul'
WHERE Pid = 5;
SELECT *FROM PATIENT;
```

Results | Explain | Describe | Saved SQL | History

PID	PNAME	AGE	GENDER
1	Amit	25	M
2	Priya	30	F
3	Vijay	45	M
4	Sneha	38	F
5	Rahul	50	M
6	Neha	28	F
7	Ajay	60	M
8	Pooja	35	F
9	Vikas	42	M
10	Anjali	22	F

5)Write a PL/SQL code to create an employee database with the tables and fields specified as below.
a)Employee[Emp no Employee name Street City]
b)Works[Emp no Company_name Joining_date Designation Salary]
c)Company[Emp no City]
d)Manages[Emp no Manager_name,Mang_no]

a)Employee[Emp no Employee name Street City]

```
CREATE TABLE Employee
(
 Empno       NUMBER(5) PRIMARY KEY,
 Empname     VARCHAR2(50),
 Street      VARCHAR2(50),
 City        VARCHAR2(30)
);
```

```
INSERT INTO Employee VALUES (1, 'Nikita', 'Bus Stand Road ', 'Bhusawal');
```

Select * from Employee

Results Explain Describe Saved SQL History

EMPNO	EMPNAME	STREET	CITY
1	Nikita	Bus Stand Road	Bhusawal

b)Works[Emp no Company_name Joining_date Designation Salary]

```
CREATE TABLE Works
(
 Empno        NUMBER(5),
 Company_name  VARCHAR2(50),
 Joining_date  DATE,
 Designation   VARCHAR2(30),
 Salary       NUMBER(10,2)
);
```

```
INSERT INTO Works VALUES (1, 'Microsoft', TO_DATE('2026-02-12','YYYY-MM-DD'),'Manager',60000);
```

select *from Works

Results Explain Describe Saved SQL History

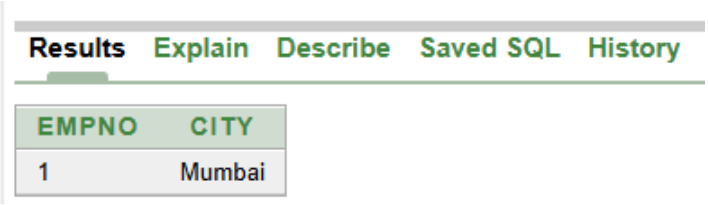
EMPNO	COMPANY_NAME	JOINING_DATE	DESIGNATION	SALARY
1	Microsoft	12-FEB-26	Manager	60000

c) Company[Emp no City]

```
CREATE TABLE Company
(
  Empno  NUMBER(5),
  City   VARCHAR2(30)
);

INSERT INTO Company VALUES (1, 'Mumbai');

select *from Company
```



Results | Explain | Describe | Saved SQL | History

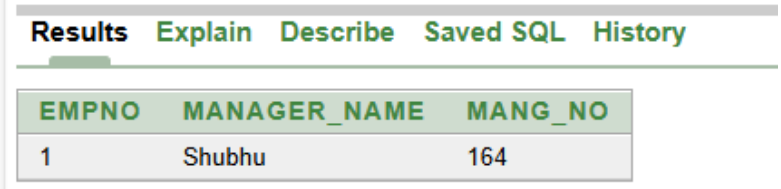
EMPNO	CITY
1	Mumbai

d)Manages[Emp no Manager_name,Mang_no]

```
CREATE TABLE Manages
(
  Empno         NUMBER(5),
  Manager_name  VARCHAR2(50),
  Mang_no       NUMBER(5)
);

INSERT INTO Manages VALUES (1, 'Shubhu',164);

select *from Manages
```



Results | Explain | Describe | Saved SQL | History

EMPNO	MANAGER_NAME	MANG_NO
1	Shubhu	164

6) PL/SQL code to retrieve the employee name, join date, and designation from employee database of an employee whose number is input by the user.

```
CREATE TABLE Employee(
Empno NUMBER PRIMARY KEY,
empname VARCHAR2(50));

INSERT INTO Employee VALUES(101,'NIKITA');
INSERT INTO Employee VALUES(102,'NEHA');
INSERT INTO Employee VALUES(103,'PRANALI');
INSERT INTO Employee VALUES(104,'NAYANA');
SELECT *FROM Employee
```

Results | Explain | Describe | Saved SQL | Hist

EMPNO	EMPNAME
101	NIKITA
102	NEHA
103	PRANALI
104	NAYANA

```
CREATE TABLE Works(
Empno NUMBER,
Joining_date DATE,
Designation VARCHAR2(50),
CONSTRAINT fk_employee_worked FOREIGN KEY(Empno) REFERENCES
Employee(Empno));

INSERT INTO Works VALUES(101,TO_DATE('2026-02-03','YYYY-MM-DD'),'Manager');
INSERT INTO Works VALUES(102,TO_DATE('2026-02-12','YYYY-MM-DD'),'HOD');
INSERT INTO Works VALUES(103,TO_DATE('2026-02-23','YYYY-MM-DD'),'Manager');
INSERT INTO Works VALUES(104,TO_DATE('2026-02-14','YYYY-MM-DD'),'HOD');
SELECT *FROM Works
```

Results | Explain | Describe | Saved SQL | History

EMPNO	JOINING_DATE	DESIGNATION
101	03-FEB-26	Manager
102	12-FEB-26	HOD
103	23-FEB-26	Manager
104	14-FEB-26	HOD

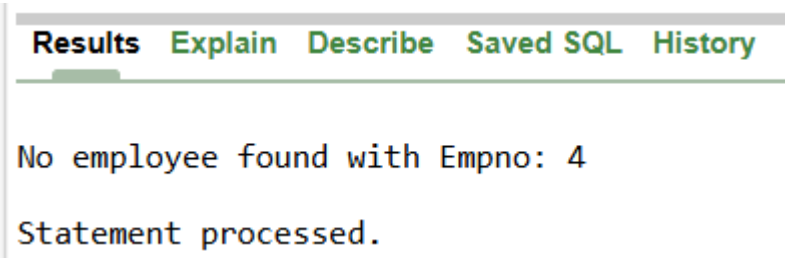
```
DECLARE
 v_empno       NUMBER;
 v_empname     Employee.Empname%TYPE;
 v_joining     Works.Joining_date%TYPE;
 v_designation Works.Designation%TYPE;
```

```
BEGIN
  v_empno := 4;

  SELECT e.Empname, w.Joining_date, w.Designation
  INTO v_empname, v_joining, v_designation
  FROM Employee e
  JOIN Works w ON e.Empno = w.Empno
  WHERE e.Empno = v_empno;

  DBMS_OUTPUT.PUT_LINE('Employee Name   : ' || v_empname);
  DBMS_OUTPUT.PUT_LINE('Joining Date    : ' || TO_CHAR(v_joining, 'DD-MON-
YYYY'));
  DBMS_OUTPUT.PUT_LINE('Designation     : ' || v_designation);

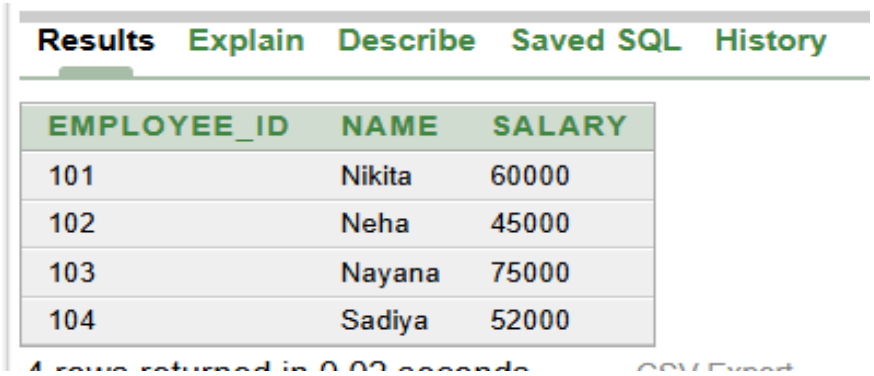
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('No employee found with Empno: ' || v_empno);
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/
```



7) Write a PL/SQL code to update the salary of employees who earn less than the average salary using cursor.

```
CREATE TABLE EMPLOYEE_DETAILS (
   EMPLOYEE_ID NUMBER,
   NAME VARCHAR2(50),
   SALARY NUMBER
);
INSERT INTO EMPLOYEE_DETAILS VALUES (101, 'Nikita', 60000);
INSERT INTO EMPLOYEE_DETAILS VALUES (102, 'Neha', 45000);
INSERT INTO EMPLOYEE_DETAILS VALUES (103, 'Nayana', 75000);
INSERT INTO EMPLOYEE_DETAILS VALUES (104, 'Sadiya', 52000);

SELECT *FROM  EMPLOYEE_DETAILS
```



Results | Explain | Describe | Saved SQL | History

EMPLOYEE_ID	NAME	SALARY
101	Nikita	60000
102	Neha	45000
103	Nayana	75000
104	Sadiya	52000

```
DECLARE
   CURSOR emp_cursor IS
      SELECT EMPLOYEE_ID, SALARY
      FROM EMPLOYEE_DETAILS;

   v_avg_salary NUMBER;
BEGIN
   -- Calculate average salary
   SELECT AVG(SALARY) INTO v_avg_salary
   FROM EMPLOYEE_DETAILS;

   FOR emp_rec IN emp_cursor LOOP
      IF emp_rec.SALARY < v_avg_salary THEN

         UPDATE EMPLOYEE_DETAILS
         SET SALARY = SALARY + 2000
         WHERE EMPLOYEE_ID = emp_rec.EMPLOYEE_ID;

      END IF;
   END LOOP;
   COMMIT;
```

```
   DBMS_OUTPUT.PUT_LINE('Salary updated for employees earning below average.');
END;
/
```

Results Explain Describe Saved SQL History

```
Salary updated for employees earning below average.

1 row(s) updated.
```

Results Explain Describe Saved SQL History

EMPLOYEE_ID	NAME	SALARY
101	Nikita	60000
102	Neha	47000
103	Nayana	75000
104	Sadiya	54000

8) Write a row trigger to insert the existing values of the salary table in to a new table when the salary table is updated.

```
CREATE TABLE Salary (
   emp_id NUMBER PRIMARY KEY,
   emp_name VARCHAR2(50),
   salary NUMBER
);
INSERT INTO Salary VALUES (101,'Nikita',40000);
INSERT INTO Salary VALUES (102,'Neha',45000);
INSERT INTO Salary VALUES (103,'Shubhu',50000);
SELECT *FROM Salary
```

Results | Explain | Describe | Saved SQL | History

EMP_ID	EMP_NAME	SALARY
101	Nikita	40000
102	Neha	45000
103	Shubhu	50000

```
CREATE TABLE salary_backup (
   emp_id NUMBER,
   emp_name VARCHAR2(50),
   old_salary NUMBER,
   update_date DATE
);
CREATE OR REPLACE TRIGGER trg_salary_update
BEFORE UPDATE ON Salary
FOR EACH ROW
BEGIN
   INSERT INTO salary_backup(emp_id, emp_name, old_salary, update_date)
   VALUES(:OLD.emp_id, :OLD.emp_name, :OLD.salary, SYSDATE);
END;
/
```

Results | Explain | Describe | Saved SQL

```
Trigger created.
```

```
UPDATE Salary
SET salary = 48000
WHERE emp_id = 102;
```

```
1 row(s) updated.
```

```
SELECT * FROM salary_backup;
```

Results | Explain | Describe | Saved SQL | History

EMP_ID	EMP_NAME	OLD_SALARY	UPDATE_DATE
102	Neha	45000	19-MAR-26

9) Write a trigger on the employee table which shows the old values and new values of Ename after any updation on Ename on Employee table.

```
CREATE TABLE Employee (
   EmpID NUMBER PRIMARY KEY,
   Ename VARCHAR2(50),
   Salary NUMBER
);
INSERT INTO Employee VALUES (101, 'John', 50000);
INSERT INTO Employee VALUES (102, 'David', 60000);
SELECT *FROM Employee
```

Results Explain Describe Saved SQL Hi

EMPID	ENAME	SALARY
101	John	50000
102	David	60000

Create Trigger to Show Old and New Ename:

```
CREATE OR REPLACE TRIGGER trg_ename_update
BEFORE UPDATE OF Ename ON Employee
FOR EACH ROW
BEGIN
   DBMS_OUTPUT.PUT_LINE('Old Ename: ' || :OLD.Ename);
   DBMS_OUTPUT.PUT_LINE('New Ename: ' || :NEW.Ename);
END;
/
```

Results Explain Describe Sa

```
Trigger created.
```

```
UPDATE Employee
SET Ename = 'Ali'
WHERE EmpID = 101;
```

Results Explain Describe Saved SQL Histo

```
Old Ename: Ali
New Ename: Shubhu

1 row(s) updated.
```

10) Write a PL/SQL procedure to find the number of students ranging from 100- 70%, 69-60%, 59-50% & below 49% in each course from the student_course table given by the procedure as parameter.

```
CREATE TABLE STUDENT_COURSE(
   SID       NUMBER PRIMARY KEY,
   SNAME     VARCHAR2(50),
   COURSE    VARCHAR2(30),
   PERCENTAGE NUMBER(5,2)
);
INSERT INTO STUDENT_COURSE VALUES (1, 'Shubhu', 'MCA', 78);
INSERT INTO STUDENT_COURSE VALUES (2, 'Neha',   'BCA', 65);
INSERT INTO STUDENT_COURSE VALUES (3, 'Nikita', 'BCA', 88);
INSERT INTO STUDENT_COURSE VALUES (4, 'Sneha',  'BSC', 55);
INSERT INTO STUDENT_COURSE VALUES (5,  'Priya', 'MBA', 91);
INSERT INTO STUDENT_COURSE VALUES (6, 'Nayana', 'MBA', 67);
INSERT INTO STUDENT_COURSE VALUES (7, 'Meena', 'MBA', 53);
INSERT INTO STUDENT_COURSE VALUES (8, 'Pranali', 'MBA', 38);
SELECT *FROM STUDENT_COURSE
```

Results | Explain | Describe | Saved SQL | History

SID	SNAME	COURSE	PERCENTAGE
1	Shubhu	MCA	78
2	Neha	BCA	65
3	Nikita	BCA	88
4	Sneha	BSC	55
5	Priya	MBA	91
6	Nayana	MBA	67
7	Meena	MBA	53
8	Pranali	MBA	38

Query:

```
CREATE OR REPLACE PROCEDURE STUDENT_PERCENT_RANGE(
   P_COURSE IN STUDENT_COURSE.COURSE%TYPE
)
IS
   V_70_100 NUMBER := 0;
   V_60_69  NUMBER := 0;
   V_50_59  NUMBER := 0;
   V_BELOW49 NUMBER := 0;
BEGIN
   SELECT
      SUM(CASE WHEN PERCENTAGE BETWEEN 70 AND 100 THEN 1 ELSE 0 END),
      SUM(CASE WHEN PERCENTAGE BETWEEN 60 AND 69 THEN 1 ELSE 0 END),
      SUM(CASE WHEN PERCENTAGE BETWEEN 50 AND 59 THEN 1 ELSE 0 END),
      SUM(CASE WHEN PERCENTAGE < 50 THEN 1 ELSE 0 END)
   INTO
      V_70_100, V_60_69, V_50_59, V_BELOW49
   FROM STUDENT_COURSE
   WHERE COURSE = P_COURSE;

   DBMS_OUTPUT.PUT_LINE('Course Name : ' || P_COURSE);
```

```
   DBMS_OUTPUT.PUT_LINE('Students (100 - 70%) : ' || V_70_100);
   DBMS_OUTPUT.PUT_LINE('Students (69 - 60%)  : ' || V_60_69);
   DBMS_OUTPUT.PUT_LINE('Students (59 - 50%)  : ' || V_50_59);
   DBMS_OUTPUT.PUT_LINE('Students (Below 49%) : ' || V_BELOW49);
EXCEPTION
   WHEN NO_DATA_FOUND THEN
      DBMS_OUTPUT.PUT_LINE('No records found for this course.');
END;
/
```




```
Procedure created.
```

Execute the Procedure:

```
BEGIN
   STUDENT_PERCENT_RANGE('MBA');
END;
/
```

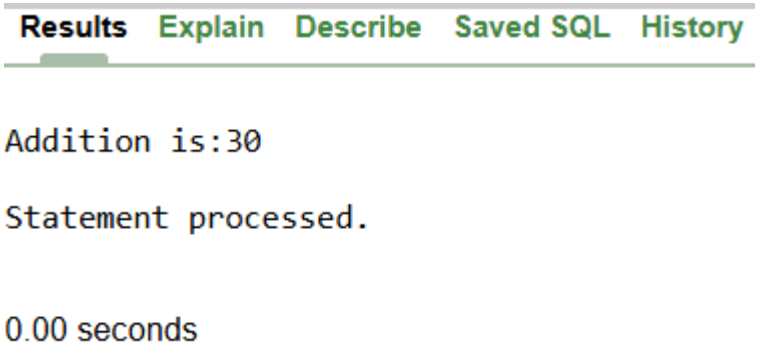



```
Course Name : MBA
Students (100 - 70%) : 1
Students (69 - 60%)  : 1
Students (59 - 50%)  : 1
Students (Below 49%) : 1

Statement processed.
```

11) Create a store function that accepts 2 numbers and returns the addition of passed values. Also, write the code to call your function**.**

```
CREATE OR REPLACE FUNCTION Addition2
(
p_num1 IN NUMBER,
p_num2 IN NUMBER
)
RETURN NUMBER
IS V_result NUMBER;
BEGIN
V_result :=p_num1+p_num2;
RETURN v_result;
END;
/
DECLARE
V_sum NUMBER;
BEGIN
v_sum:=Addition2(10,20);
DBMS_OUTPUT.PUT_LINE('Addition is:'||v_sum);
END;
/
```



12) Write a PL/SQL function that accepts the department number and returns the total salary of the department. Also, write a function to call the function.

```
CREATE TABLE EmployeeDept(
emp_id NUMBER PRIMARY KEY,
emp_name VARCHAR2(50),
dept_no NUMBER,
salary NUMBER(10,2)
);
INSERT INTO EmployeeDept VALUES(101,'NIKITA',1,40000);
INSERT INTO EmployeeDept VALUES(102,'NAYANA',2,25000);
INSERT INTO EmployeeDept VALUES(103,'PRANALI',3,25000);
INSERT INTO EmployeeDept VALUES(104,'SADIYA',4,30000);
INSERT INTO EmployeeDept VALUES(105,'NIDA',5,35000);
INSERT INTO EmployeeDept VALUES(106,'SHUBHU',6,50000);

SELECT *FROM EmployeeDept
```

Results | Explain | Describe | Saved SQL | History

EMP_ID	EMP_NAME	DEPT_NO	SALARY
101	NIKITA	1	40000
102	NAYANA	2	25000
103	PRANALI	3	25000
104	SADIYA	4	30000
105	NIDA	5	35000
106	SHUBHU	6	50000

6 rows returned in 0.00 seconds CSV Export

```
CREATE OR REPLACE FUNCTION get_total_salary(
p_dept_no IN NUMBER
)
RETURN NUMBER
IS
v_total_salary NUMBER:=0;
BEGIN
SELECT SUM(salary)
INTO v_total_salary
FROM EmployeeDept
where dept_no=p_dept_no;
RETURN NVL(v_total_salary,0);
RETURN 0;
END;
/
```

Results Explain Describe Saved SQL History

Function created.

0.02 seconds

```
DECLARE
v_total NUMBER;
BEGIN
v_total:=get_total_salary(10);
DBMS_OUTPUT.PUT_LINE('total salary of department 10:'||v_total);
END;
/
```

OUTPUT*****

Results Explain Describe Saved SQL History

```
total salary of department 10:0

Statement processed.
```

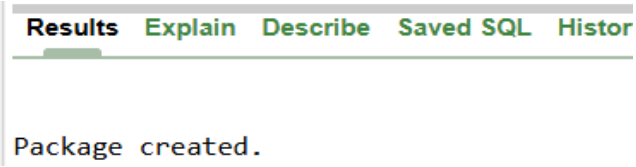
0.00 seconds

13) Write a PL/SQL code to create,
1. Package specification
2. Package body.
For the insert, retrieve, update, and delete operations on a student table.

```
CREATE TABLE STUDENT (
   STUDENT_ID NUMBER PRIMARY KEY,
   NAME VARCHAR2(50),
   COURSE VARCHAR2(50),
   MARKS NUMBER
);
```

Package Specification:

```
CREATE OR REPLACE PACKAGE student_pkg AS
   PROCEDURE insert_student(
      p_id NUMBER,
      p_name VARCHAR2,
      p_course VARCHAR2,
      p_marks NUMBER
   );
   PROCEDURE get_student(
      p_id NUMBER
   );
   PROCEDURE update_student(
      p_id NUMBER,
      p_marks NUMBER
   );
   PROCEDURE delete_student(
      p_id NUMBER
   );
END student_pkg;
/
```



Package Body:

```
CREATE OR REPLACE PACKAGE BODY student_pkg AS
-- Insert Operation
PROCEDURE insert_student(
   p_id NUMBER,
   p_name VARCHAR2,
   p_course VARCHAR2,
   p_marks NUMBER
) IS
BEGIN
   INSERT INTO STUDENT VALUES(p_id, p_name, p_course, p_marks);
   DBMS_OUTPUT.PUT_LINE('Student inserted successfully');
END insert_student;
-- Retrieve Operation
PROCEDURE get_student(p_id NUMBER) IS
```

```
    v_name STUDENT.NAME%TYPE;
    v_course STUDENT.COURSE%TYPE;
    v_marks STUDENT.MARKS%TYPE;
BEGIN
    SELECT NAME, COURSE, MARKS
    INTO v_name, v_course, v_marks
    FROM STUDENT
    WHERE STUDENT_ID = p_id;
    DBMS_OUTPUT.PUT_LINE('Name: ' || v_name);
    DBMS_OUTPUT.PUT_LINE('Course: ' || v_course);
    DBMS_OUTPUT.PUT_LINE('Marks: ' || v_marks);
END get_student;
-- Update Operation
PROCEDURE update_student(
    p_id NUMBER,
    p_marks NUMBER
) IS
BEGIN
    UPDATE STUDENT
    SET MARKS = p_marks
    WHERE STUDENT_ID = p_id;
    DBMS_OUTPUT.PUT_LINE('Student record updated');
END update_student;
-- Delete Operation
PROCEDURE delete_student(p_id NUMBER) IS
BEGIN
    DELETE FROM STUDENT
    WHERE STUDENT_ID = p_id;
    DBMS_OUTPUT.PUT_LINE('Student record deleted');
END delete_student;
END student_pkg;
/
```




```
Package Body created.
```

Calling the Package:

```
BEGIN
    student_pkg.insert_student(102,'Shubhu','BCA',85);
    student_pkg.insert_student(103,'Nikita','BCA',84);
END;
/
```




```
Student inserted successfully
Student inserted successfully

Statement processed.
```

```
BEGIN
   student_pkg.get_student(102);
   student_pkg.get_student(103);
END;
/
```

Results Explain Describe Saved SQL Histor

```
Name: Shubhu
Course: BSC
Marks: 85
Name: Nikita
Course: BCA
Marks: 84

Statement processed.
```

```
BEGIN
   student_pkg.update_student(101,90);
END;
/
```

Results Explain Describe Saved SQL Histo

```
Student record updated

Statement processed.
```

```
BEGIN
   student_pkg.delete_student(101);
END;
/
```

Results Explain Describe Saved SQL Histor

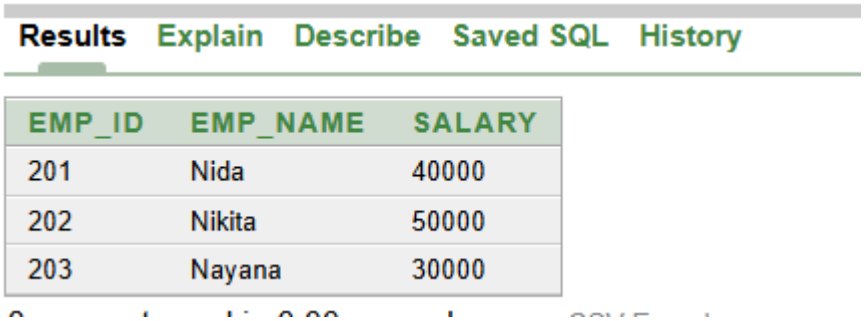
```
Student record deleted

Statement processed.
```

14) Write a program to illustrate user-defined exceptions, built-in exceptions, and raise application error exceptions.

```
CREATE TABLE emp_demo(
emp_id NUMBER PRIMARY KEY ,
emp_name VARCHAR2(50),
salary NUMBER(10,2)
);
INSERT INTO emp_demo VALUES(201,'Nida',40000);
INSERT INTO emp_demo VALUES(202,'Nikita',50000);
INSERT INTO emp_demo VALUES(203,'Nayana',30000);

SELECT *FROM emp_demo
```



Results Explain Describe Saved SQL History

EMP_ID	EMP_NAME	SALARY
201	Nida	40000
202	Nikita	50000
203	Nayana	30000

```
DECLARE
v_salary emp_demo.salary%TYPE;
v_emp_id NUMBER:=203;
v_bonus NUMBER;
low_salary EXCEPTION;
BEGIN
SELECT salary
INTO v_salary
FROM emp_demo
WHERE emp_id=v_emp_id;

DBMS_OUTPUT.PUT_LINE('salary is:'||v_salary);

IF v_salary<40000 THEN
RAISE low salary;
END IF;
IF v_salary>100000 THEN
RAISE_APPLICATION_ERROR(-20001,'Salary exceeds allowed limit');
END IF;
v_bonus:=v_salary*0.10;
DBMS_OUTPUT.PUT_LINE('Bonus is:'||v_bonus);
EXCEPTION
WHEN NO_DATA_FOUND THEN
```

```
DBMS_OUTPUT.PUT_LINE('Error:Employee not found');

WHEN low_salary THEN
DBMS_OUTPUT.PUT_LINE('Error:salary is less than 40,000.');

WHEN OTHERS THEN
DBMS_OUTPUT.PUT_LINE('Other Error:'||SQLERRM);
END;
/
```



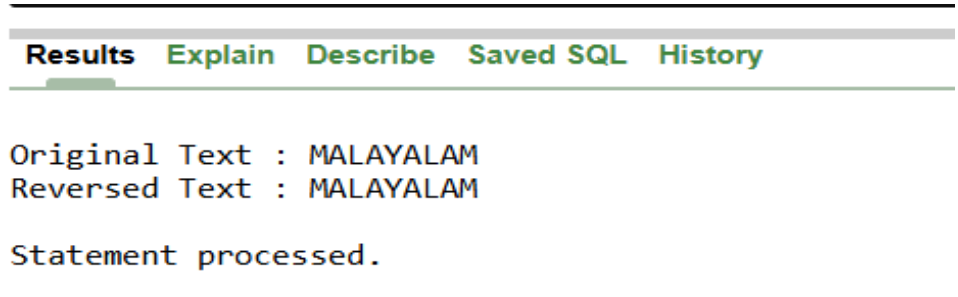

```
salary is:50000
Bonus is:5000

Statement processed.
```

15) Write a program Reversing a String Using PL/SQL Block.

```
DECLARE
   original_data VARCHAR2(100) := 'MALAYALAM';
   reversed_data VARCHAR2(100) := '';
BEGIN
   -- Loop through the string in reverse order
   FOR i IN REVERSE 1 .. LENGTH(original_data) LOOP
      reversed_data := reversed_data || SUBSTR(original_data, i, 1);
   END LOOP;

   -- Display the result
   DBMS_OUTPUT.PUT_LINE('Original Text : ' || original_data);
   DBMS_OUTPUT.PUT_LINE('Reversed Text : ' || reversed_data);
END;
/
```



16) Trigger for Auditing Table Changes

- **Create a trigger that records changes to an EMPLOYEES table (INSERT, UPDATE, DELETE) into an EMPLOYEES_AUDIT table. Include details like EMPLOYEE_ID, OPERATION_TYPE, TIMESTAMP.**

```
CREATE TABLE EMPLOYEES (
   EMPLOYEE_ID NUMBER PRIMARY KEY,
   ENAME VARCHAR2(50),
   SALARY NUMBER
);
CREATE TABLE EMPLOYEES_AUDIT (
   EMPLOYEE_ID NUMBER,
   OPERATION_TYPE VARCHAR2(10),
   ACTION_DATE TIMESTAMP);
```

Create Trigger for Auditing:

```
CREATE OR REPLACE TRIGGER trg_employees_audit
AFTER INSERT OR UPDATE OR DELETE ON EMPLOYEES
FOR EACH ROW
BEGIN
   IF INSERTING THEN
      INSERT INTO EMPLOYEES_AUDIT
      VALUES (:NEW.EMPLOYEE_ID, 'INSERT', SYSTIMESTAMP);

   ELSIF UPDATING THEN
      INSERT INTO EMPLOYEES_AUDIT
      VALUES (:NEW.EMPLOYEE_ID, 'UPDATE', SYSTIMESTAMP);

   ELSIF DELETING THEN
      INSERT INTO EMPLOYEES_AUDIT
      VALUES (:OLD.EMPLOYEE_ID, 'DELETE', SYSTIMESTAMP);
   END IF;
END;
/
```

Results Explain Describe Saved SQL His

```
Trigger created.
```

Test the Trigger:

```
-- Insert
INSERT INTO EMPLOYEES VALUES (1, 'John', 50000);
INSERT INTO EMPLOYEES VALUES (2, 'Nikita', 50000);
-- Update
UPDATE EMPLOYEES SET SALARY = 55000 WHERE EMPLOYEE_ID = 1;
-- Delete
DELETE FROM EMPLOYEES WHERE EMPLOYEE_ID = 1;
SELECT * FROM EMPLOYEES_AUDIT;
```

Results Explain Describe Saved SQL History

EMPLOYEE_ID	OPERATION_TYPE	ACTION_DATE
1	INSERT	19-MAR-26 03.35.23.599000 PM
2	INSERT	19-MAR-26 03.36.25.983000 PM
1	UPDATE	19-MAR-26 03.36.53.196000 PM
1	DELETE	19-MAR-26 03.37.10.348000 PM

17) Employee Bonus Calculation Using Cursor

- **Write a PL/SQL program using an explicit cursor to calculate and display a 10% bonus for all employees whose salary is greater than 50,000. Assume a table EMPLOYEES with columns EMPLOYEE_ID, NAME, and SALARY.**

```
CREATE TABLE Employee_Details(
Emp_id NUMBER,
Ename VARCHAR2(50),
Salary NUMBER);

INSERT INTO Employee_Details VALUES(101,'NIKITA',60000);
INSERT INTO Employee_Details VALUES(102,'NEHA',70000);
INSERT INTO Employee_Details VALUES(103,'NAYANA',50000);
INSERT INTO Employee_Details VALUES(104,'PRANALI',40000);
INSERT INTO Employee_Details VALUES(105,'PAYAL',30000);
INSERT INTO Employee_Details VALUES(106,'PRIYA',40000);
SELECT *FROM Employee_Details;
```

Results | Explain | Describe | Saved SQL | History

EMP_ID	ENAME	SALARY
101	NIKITA	60000
102	NEHA	70000
103	NAYANA	50000
104	PRANALI	40000
105	PAYAL	30000
106	PRIYA	40000

```
DECLARE
  CURSOR emp_cursor IS
    SELECT Emp_id,Ename,Salary
    FROM Employee_Details
    WHERE Salary>50000;
  v_id Employee_Details.Emp_id%TYPE;
  v_name Employee_Details.Ename%TYPE;
  v_salary Employee_Details.Salary%TYPE;
  v_bonus NUMBER;
BEGIN
  OPEN emp_cursor;
  LOOP
    FETCH emp_cursor INTO v_id,v_name,v_salary;
    EXIT WHEN emp_cursor%NOTFOUND;
    v_bonus := v_salary*0.10;
    DBMS_OUTPUT.PUT_LINE('Employee ID:'||v_id);
    DBMS_OUTPUT.PUT_LINE('Name:'||v_name);
    DBMS_OUTPUT.PUT_LINE('Salary:'||v_salary);
    DBMS_OUTPUT.PUT_LINE('Bnus:'||v_bonus);
```

```
    DBMS_OUTPUT.PUT_LINE('------------------------');
  END LOOP;
  CLOSE emp_cursor;
END;
/
```

Results Explain Describe Saved SQL History

```
Employee ID:101
Name:NIKITA
Salary:60000
Bnus:6000
------------------------
Employee ID:102
Name:NEHA
Salary:70000
Bnus:7000
------------------------

Statement processed.
```

18)Write a SQL Program to implement Aggregate Functions.

```
CREATE TABLE EmployeeDetail (
   Emp_ID NUMBER,
   Emp_Name VARCHAR2(20),
   Salary NUMBER,
   Department VARCHAR2(20)
);

INSERT INTO EmployeeDetail VALUES (1, 'Abuzar', 30000, 'HR');
INSERT INTO EmployeeDetail  VALUES (2, 'Ubaid', 45000, 'IT');
INSERT INTO EmployeeDetail  VALUES (3, 'Moin', 40000, 'IT');
INSERT INTO EmployeeDetail  VALUES (4, 'Shahid', 35000, 'HR');
INSERT INTO EmployeeDetail  VALUES (5, 'Rehan', 50000, 'Finance');

SELECT *FROM EmployeeDetail
```

Results | Explain | Describe | Saved SQL | History

EMP_ID	EMP_NAME	SALARY	DEPARTMENT
1	Shubhu	30000	HR
2	Nikita	45000	IT
3	Neha	40000	IT
4	Jayesh	35000	HR
5	Priya	50000	Finance

Count()

```
SELECT COUNT(*) AS Total_Employees
FROM EmployeeDetail;
```

Results | Explain | Describe | Saved SQL | History

TOTAL_EMPLOYEES
5

SUM()

```
SELECT SUM(Salary) AS Total_Salary
FROM EmployeeDetail;
```

Results | Explain | Describe | Saved SQL | History

TOTAL_SALARY
200000

AVG()

```
SELECT AVG(Salary) AS Average_Salary
FROM EmployeeDetail;
```

Results Explain Describe Saved SQL History

AVERAGE_SALARY
40000

□MAX()

```
SELECT MAX(Salary) AS Highest_Salary
FROM EmployeeDetail;
```

Results Explain Describe Saved SQL History

HIGHEST_SALARY
50000

□MIN()

```
SELECT MIN(Salary) AS Lowest_Salary
FROM EmployeeDetail;
```

Results Explain Describe Saved SQL History

LOWEST_SALARY
30000

□Aggregate Functions with GROUP BY

```
SELECT Department, COUNT(*) AS Emp_Count, AVG(Salary) AS Avg_Salary
FROM EmployeeDetail
GROUP BY Department;
```

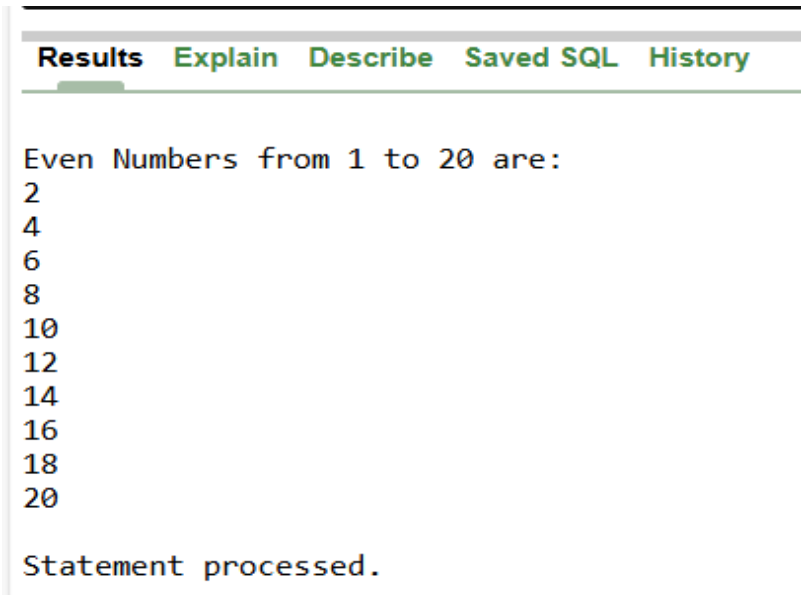
Results Explain Describe Saved SQL History

DEPARTMENT	EMP_COUNT	AVG_SALARY
IT	2	42500
HR	2	32500
Finance	1	50000

19) Write PL/SQL code for finding Even Numbers.

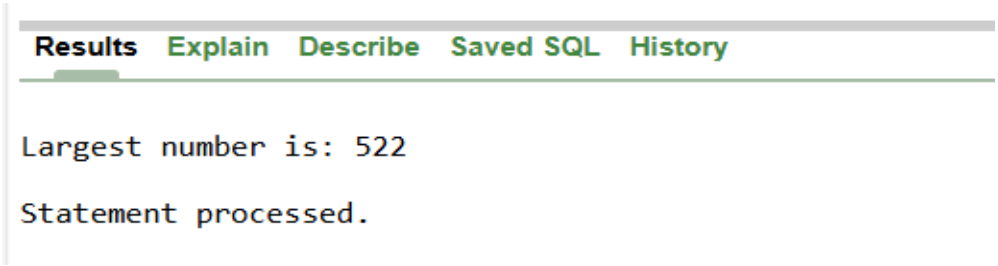
```
DECLARE
   i NUMBER;
BEGIN
   DBMS_OUTPUT.PUT_LINE('Even Numbers from 1 to 20 are:');

   FOR i IN 1..20 LOOP
      IF MOD(i, 2) = 0 THEN
         DBMS_OUTPUT.PUT_LINE(i);
      END IF;
   END LOOP;
END;
/
```



20) Write PL/SQL code to find Largest of three numbers.

```
DECLARE
   a NUMBER := 164;
   b NUMBER := 522;
   c NUMBER := 213;
   largest NUMBER;
BEGIN
   IF a >= b AND a >= c THEN
      largest := a;
   ELSIF b >= a AND b >= c THEN
      largest := b;
   ELSE
      largest := c;
   END IF;
   DBMS_OUTPUT.PUT_LINE('Largest number is: ' || largest);
END;
/
```



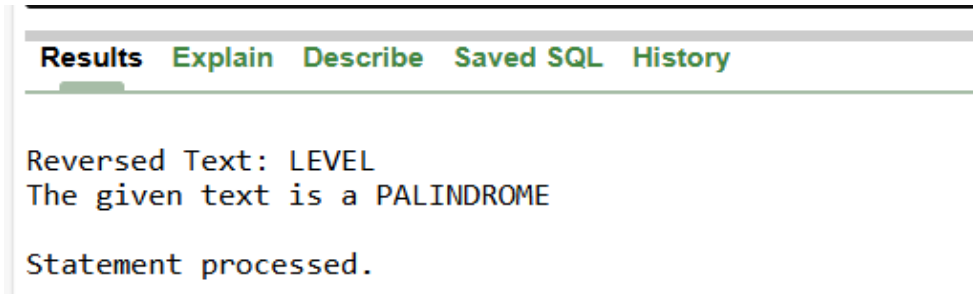
21) Write PL/SQL code to accept the text and reverse the text and test whether the given character is Palindrome or not.

```
DECLARE
   input_text   VARCHAR2(100);
   reversed_text VARCHAR2(100) := '';
   i NUMBER;
BEGIN
   input_text := 'LEVEL';

   FOR i IN REVERSE 1..LENGTH(input_text) LOOP
      reversed_text := reversed_text || SUBSTR(input_text, i, 1);
   END LOOP;

   DBMS_OUTPUT.PUT_LINE('Reversed Text: ' || reversed_text);


   IF input_text = reversed_text THEN
      DBMS_OUTPUT.PUT_LINE('The given text is a PALINDROME');
   ELSE
      DBMS_OUTPUT.PUT_LINE('The given text is NOT a PALINDROME');
   END IF;
END;
/
```



22) Write PL/SQL code to Insert values in created tables.

```
CREATE TABLE emp(
emp_id NUMBER,
ename VARCHAR2(50),
salary NUMBER);
INSERT INTO emp VALUES(101,'Nikita',30000);
INSERT INTO emp VALUES(102,'Nayana',40000);
INSERT INTO emp VALUES(103,'Neha',50000);
INSERT INTO emp VALUES(104,'Pranali',30000);
INSERT INTO emp VALUES(105,'Shubhu',60000);
SELECT *FROM emp;
```

Results | Explain | Describe | Saved SQL | History

EMP_ID	ENAME	SALARY
101	Nikita	30000
102	Nayana	40000
103	Neha	50000
104	Pranali	30000
105	Shubhu	60000

```
BEGIN
  INSERT INTO emp VALUES(106,'Sadiya',60000);
  INSERT INTO emp VALUES(107,'Nida',70000);
  INSERT INTO emp VALUES(108,'Payal',80000);
  IF SQL%FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Record Inserted Successfully');
  ELSE
    DBMS_OUTPUT.PUT_LINE('Insertion Failed');
  END IF;
END;
/
```

Results | Explain | Describe | Saved SQL | History

```
Record Inserted Successfully

1 row(s) inserted.
```

```
SELECT *FROM emp
```

Results | Explain | Describe | Saved SQL | History

EMP_ID	ENAME	SALARY
101	Nikita	38000
102	Nayana	40000
103	Neha	50000
104	Pranali	30000
105	Shubhu	60000
106	Sadiya	60000
107	Nida	70000
108	Payal	80000

23) Write PL/SQL code to UPDATE values in created tables by using Implicit Cursors

```
CREATE TABLE emp(
emp_id NUMBER,
ename VARCHAR2(50),
salary NUMBER);
INSERT INTO emp VALUES(101,'Nikita',30000);
INSERT INTO emp VALUES(102,'Nayana',40000);
INSERT INTO emp VALUES(103,'Neha',50000);
INSERT INTO emp VALUES(104,'Pranali',30000);
INSERT INTO emp VALUES(105,'Shubhu',60000);
SELECT *FROM emp;
```

Results | Explain | Describe | Saved SQL | History

EMP_ID	ENAME	SALARY
101	Nikita	30000
102	Nayana	40000
103	Neha	50000
104	Pranali	30000
105	Shubhu	60000

```
BEGIN
  UPDATE emp
  SET salary = salary+8000
  WHERE emp_id = 101;
  IF SQL%FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Record Updated Successfully');
  ELSE
    DBMS_OUTPUT.PUT_LINE('Record Not Found');
  END IF;
END;
/
```

Results | Explain | Describe | Saved SQL | History

```
Record Updated Successfully

1 row(s) updated.
```

```
SELECT *FROM emp
```

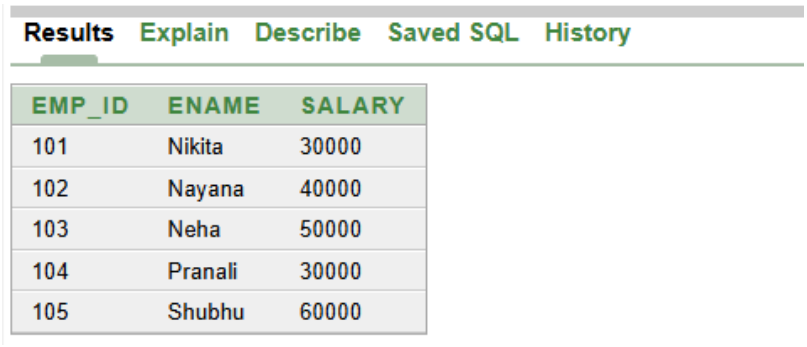
Results | Explain | Describe | Saved SQL | History

EMP_ID	ENAME	SALARY
101	Nikita	38000
102	Nayana	40000
103	Neha	50000
104	Pranali	30000
105	Shubhu	60000

24) Write PL/SQL code to display Employee details using Explicit Cursors

```
CREATE TABLE emp(
emp_id NUMBER,
ename VARCHAR2(50),
salary NUMBER);

INSERT INTO emp VALUES(101,'Nikita',30000);
INSERT INTO emp VALUES(102,'Nayana',40000);
INSERT INTO emp VALUES(103,'Neha',50000);
INSERT INTO emp VALUES(104,'Pranali',30000);
INSERT INTO emp VALUES(105,'Shubhu',60000);
SELECT *FROM emp;
```



Results | Explain | Describe | Saved SQL | History

EMP_ID	ENAME	SALARY
101	Nikita	30000
102	Nayana	40000
103	Neha	50000
104	Pranali	30000
105	Shubhu	60000

```
DECLARE
  CURSOR emp_cursor IS
   SELECT emp_id, ename, salary FROM emp;
  v_id emp.emp_id%TYPE;
  v_name emp.ename%TYPE;
  v_salary emp.salary%TYPE;
BEGIN
  OPEN emp_cursor;
  LOOP
    FETCH emp_cursor INTO v_id, v_name, v_salary;
    EXIT WHEN emp_cursor%NOTFOUND;

    DBMS_OUTPUT.PUT_LINE('Employee ID:'||v_id);
    DBMS_OUTPUT.PUT_LINE('Employee Name:'||v_name);
    DBMS_OUTPUT.PUT_LINE('Salary:'||v_salary);
    DBMS_OUTPUT.PUT_LINE('-----------------------------');
  END LOOP;
  CLOSE emp_cursor;
END;
/
```

Results Explain Describe Saved SQL History

```
Employee ID:101
Employee Name:Nikita
Salary:30000
------------------------------
Employee ID:102
Employee Name:Nayana
Salary:40000
------------------------------
Employee ID:103
Employee Name:Neha
Salary:50000
------------------------------
Employee ID:104
Employee Name:Pranali
Salary:30000
------------------------------
Employee ID:105
Employee Name:Shubhu
Salary:60000
------------------------------

Statement processed.
```

25) Write PL/SQL code in Cursor to display employee names and salary.

```
CREATE TABLE emp(
emp_id NUMBER,
ename VARCHAR2(50),
salary NUMBER);

INSERT INTO emp VALUES(101,'Nikita',30000);
INSERT INTO emp VALUES(102,'Nayana',40000);
INSERT INTO emp VALUES(103,'Neha',50000);
INSERT INTO emp VALUES(104,'Pranali',30000);
INSERT INTO emp VALUES(105,'Shubhu',60000);
SELECT *FROM emp;
```

Results | Explain | Describe | Saved SQL | History

EMP_ID	ENAME	SALARY
101	Nikita	30000
102	Nayana	40000
103	Neha	50000
104	Pranali	30000
105	Shubhu	60000

```
DECLARE
 CURSOR emp_cursor IS
  SELECT ename, salary FROM emp;
 v_name emp.ename%TYPE;
 v_salary emp.salary%TYPE;
BEGIN
 OPEN emp_cursor;
 LOOP
  FETCH emp_cursor INTO v_name,v_salary;
  EXIT WHEN emp_cursor%NOTFOUND;
  DBMS_OUTPUT.PUT_LINE('Employee Name:' ||v_name|| ' ,Salary:' ||v_salary);
 END LOOP;
 CLOSE emp_cursor;
END;
/
```

Results | Explain | Describe | Saved SQL | History

```
Employee Name:Nikita ,Salary:30000
Employee Name:Nayana ,Salary:40000
Employee Name:Neha ,Salary:50000
Employee Name:Pranali ,Salary:30000
Employee Name:Shubhu ,Salary:60000

Statement processed.
```

26) Write PL/SQL Programs in Cursors using two cursors at a time.

```
CREATE TABLE department(
   dept_id NUMBER,
   dept_name VARCHAR2(50)
);
CREATE TABLE employee(
   emp_id NUMBER,
   emp_name VARCHAR2(50),
   salary NUMBER,
   dept_id NUMBER
);
```

Insert values into the table: first table values-

```
INSERT INTO department VALUES (10,'HR');
INSERT INTO department VALUES (20,'IT');
SELECT *FROM department;
```

Second table values-

```
INSERT INTO employee VALUES (101,'Amit',20000,10);
INSERT INTO employee VALUES (102,'Rahul',25000,10);
INSERT INTO employee VALUES (103,'Sneha',22000,20);
INSERT INTO employee VALUES (104,'Priya',27000,20);
SELECT *FROM employee;
```

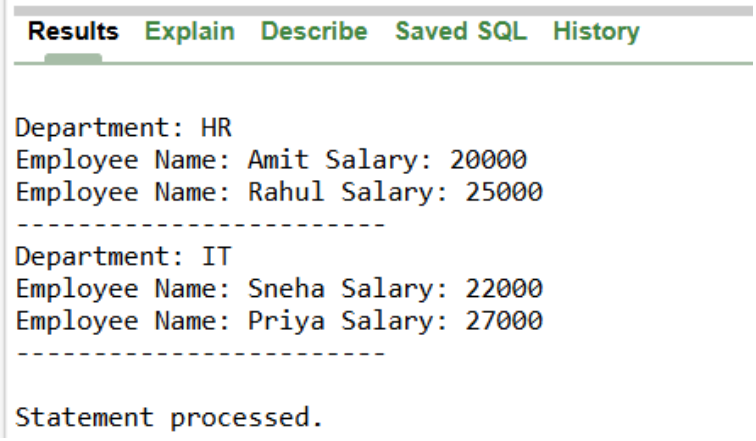
Cursor Query:

```
DECLARE
CURSOR dept_cursor IS
SELECT dept_id, dept_name FROM department;

CURSOR emp_cursor(d_id NUMBER) IS
SELECT emp_name, salary FROM employee WHERE dept_id = d_id;

v_dept_id department.dept_id%TYPE;
v_dept_name department.dept_name%TYPE;
v_emp_name employee.emp_name%TYPE;
v_salary employee.salary%TYPE;
BEGIN
OPEN dept_cursor;
LOOP
   FETCH dept_cursor INTO v_dept_id, v_dept_name;
   EXIT WHEN dept_cursor%NOTFOUND;
   DBMS_OUTPUT.PUT_LINE('Department: ' || v_dept_name);
   OPEN emp_cursor(v_dept_id);
   LOOP
      FETCH emp_cursor INTO v_emp_name, v_salary;
      EXIT WHEN emp_cursor%NOTFOUND;
      DBMS_OUTPUT.PUT_LINE('Employee Name: ' || v_emp_name || ' Salary: ' ||
v_salary);
   END LOOP;
   CLOSE emp_cursor;
   DBMS_OUTPUT.PUT_LINE('------------------------');
```

```
END LOOP;
CLOSE dept_cursor;
END;
/
```



27) Write PL/SQL code in Procedure to find Reverse number

```
CREATE OR REPLACE PROCEDURE REVERSE_NUMBER
(
   P_NUM IN NUMBER
)
IS
   V_NUM     NUMBER := P_NUM;
   V_REVERSE NUMBER := 0;
   V_REM     NUMBER;
BEGIN
   WHILE V_NUM > 0 LOOP
      V_REM := MOD(V_NUM, 10);
      V_REVERSE := (V_REVERSE * 10) + V_REM;
      V_NUM := TRUNC(V_NUM / 10);
   END LOOP;

   DBMS_OUTPUT.PUT_LINE('Original Number : ' || P_NUM);
   DBMS_OUTPUT.PUT_LINE('Reverse Number  : ' || V_REVERSE);
END;
/
```

Results Explain Describe Saved SQL History

```
Procedure created.
```

Procedure Execution:

```
BEGIN
   REVERSE_NUMBER(12345);
END;
/
```

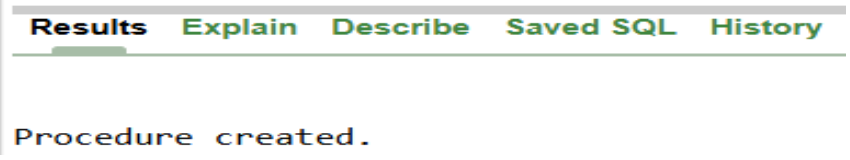
Results Explain Describe Saved SQL History

```
Original Number : 12345
Reverse Number  : 54321

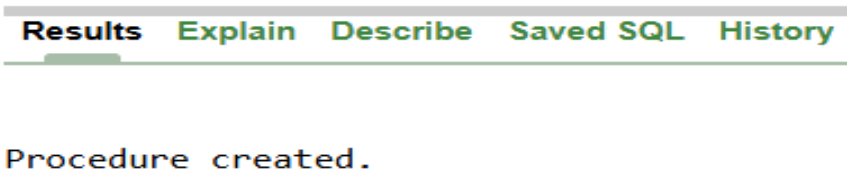
Statement processed.
```

28) Write PL/SQL code in Procedure to find Factorial of a given number by using call Procedure.

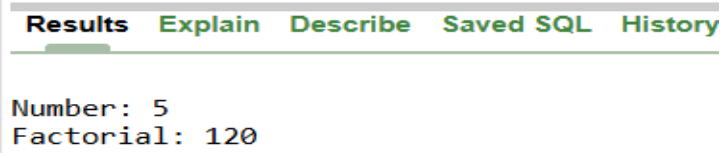
```
CREATE OR REPLACE PROCEDURE CALS_FACTORIAL(
P_NUM IN NUMBER,
P_FACT OUT NUMBER)
IS
V_FACT NUMBER:=1;
BEGIN
IF P_NUM <0 THEN
P_FACT := NULL;
ELSE
FOR i IN 1..P_NUM LOOP
V_FACT := V_FACT*i;
END LOOP;
P_FACT := V_FACT;
END IF;
END;
/
```




```
CREATE OR REPLACE PROCEDURE fact_prac(
P_NUM IN NUMBER)
IS
V_RESULT NUMBER;
BEGIN
CALS_FACTORIAL(P_NUM,V_RESULT);
DBMS_OUTPUT.PUT_LINE('Number: '||P_NUM);
DBMS_OUTPUT.PUT_LINE('Factorial: '||V_RESULT);
END;
/
```




```
BEGIN
fact_prac(5);
END;
/
```



29) Write a procedure to retrieve the salary of a particular employee

```
CREATE TABLE Employees (
   emp_id     NUMBER PRIMARY KEY,
   emp_name   VARCHAR2(50),
   salary     NUMBER(10,2),
   department VARCHAR2(50)
);
```

Insert values into the table.

```
INSERT INTO employees VALUES (101, 'Amit',   35000, 'HR');
INSERT INTO employees VALUES (102, 'Rahul',  42000, 'IT');
INSERT INTO employees VALUES (103, 'Sneha',  39000, 'Finance');
INSERT INTO employees VALUES (104, 'Pooja',  28000, 'Marketing');
INSERT INTO employees VALUES (105, 'Karan',  50000, 'IT');
SELECT *FROM Employees;
```

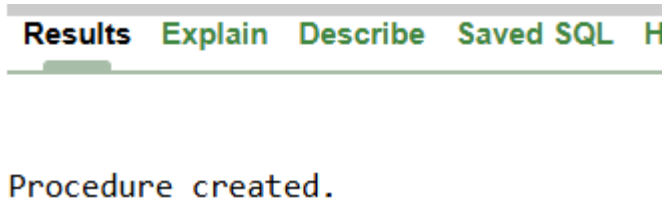
Results | Explain | Describe | Saved SQL | History

EMP_ID	EMP_NAME	SALARY	DEPARTMENT
101	Nikita	35000	HR
102	Rahul	42000	IT
103	Priya	39000	Finance
104	Neha	28000	Marketing
105	Akshu	50000	IT

Store Procedure:

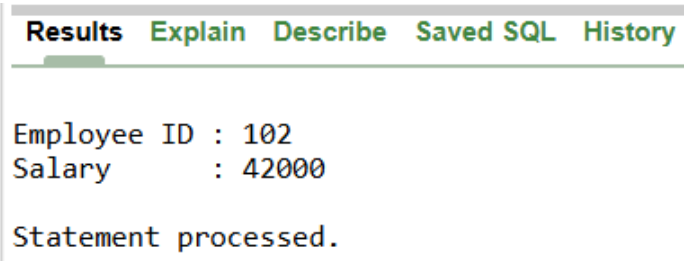
```
CREATE OR REPLACE PROCEDURE get_employee_salary(
   p_emp_id IN employees.emp_id%TYPE
)
IS
   v_salary employees.salary%TYPE;
BEGIN
   SELECT salary
   INTO v_salary
   FROM employees
   WHERE emp_id = p_emp_id;

   DBMS_OUTPUT.PUT_LINE('Employee ID : ' || p_emp_id);
   DBMS_OUTPUT.PUT_LINE('Salary      : ' || v_salary);
EXCEPTION
   WHEN NO_DATA_FOUND THEN
      DBMS_OUTPUT.PUT_LINE('Employee not found.');
   WHEN TOO_MANY_ROWS THEN
      DBMS_OUTPUT.PUT_LINE('Multiple employees found with same ID.');
END;
/
```



Procedure Execution:

```
BEGIN
  get_employee_salary(102);
END;
/
```



30) Write PL/SQL code in Trigger not to accept the existing Empno (Unique no)

```
CREATE TABLE Employee (
   Empno NUMBER,
   Ename VARCHAR2(50),
   Salary NUMBER
);
```

Create Trigger to Restrict Duplicate Empno

```
CREATE OR REPLACE TRIGGER trg_unique_empno
BEFORE INSERT ON Employee
FOR EACH ROW
DECLARE
   v_count NUMBER;
BEGIN
   -- Check if Empno already exists
   SELECT COUNT(*)
   INTO v_count
   FROM Employee
   WHERE Empno = :NEW.Empno;

   -- If exists, raise error
   IF v_count > 0 THEN
      RAISE_APPLICATION_ERROR(-20001, 'Empno already exists. Duplicate not
allowed.');
   END IF;
END;
/
```

Results Explain Describe Saved SQL Hi

```
Trigger created.
```

Test the Trigger:

```
-- First insert (works)
INSERT INTO Employee VALUES (101, 'John', 50000);
-- Duplicate insert (fails)
INSERT INTO Employee VALUES (101, 'Niku', 60000);
```

Results Explain Describe Saved SQL History

```
ORA-20001: Empno already exists. Duplicate not allowed.
ORA-06512: at "SCOTT.TRG_UNIQUE_EMPNO", line 12
ORA-04088: error during execution of trigger 'SCOTT.TRG_UNIQUE_EMPNO'
1. INSERT INTO Employee VALUES (101, 'Ali', 60000);
```

SELECT *FROM Employee

Results Explain Describe Saved SQL History

EMPNO	ENAME	SALARY
101	John	50000